Capitolo 6

Java funzionale, lambda, collection e stream

6.1 Dallo stile imperativo allo stile dichiarativo

Java è un linguaggio imperativo: in questo stile il programmatore descrive esplicitamente i passi da eseguire, per esempio il ciclo che attraversa una lista e la variabile che accumula un risultato. A partire da Java 8 il linguaggio include caratteristiche della **programmazione funzionale**, in genere più concise, espressive e più facilmente parallelizzabili.

Lo stile funzionale è dichiarativo: si esprime *che cosa* si vuole ottenere, delegando alla libreria i dettagli dell'iterazione. Non sostituisce la programmazione a oggetti, ma la affianca.

Per esempio, cercare un nome con un ciclo esterno richiede un flag booleano e la gestione dell'uscita dal ciclo; la stessa intenzione dichiarativa si esprime con `nomi.contains("Nobi")`. Nel secondo caso l'iterazione è *interna* all'implementazione della lista.

Gli stream estendono questa idea: consentono di descrivere una successione di trasformazioni e selezioni sugli elementi.

Il confronto completo della ricerca è:

```
List<String> nomi = List.of("Nobita", "Nobi", "Suneo", "Honekawa",
    "Shizuka", "Minamoto", "Takeshi", "Gouda");
boolean trovato = false;
for (String nome : nomi) {
    if (nome.equals("Nobi")) { trovato = true; break; }
}
boolean interno = nomi.contains("Nobi");
```

Entrambe le variabili risultano vere. Nel ciclo il programmatore controlla il flag e l'uscita; `contains` concentra nella collezione la stessa richiesta. Questo è un confronto fra controllo dell'iterazione e intenzione dichiarata, non una promessa che i due modi abbiano prestazioni diverse.

Le principali tappe del linguaggio comprendono classi anonime da Java 1.1, enhanced for da Java 5, diamond `<>` da Java 7 e lambda e stream da Java 8. Il contesto è anche l'uso di processori con più core: esprimere trasformazioni e selezioni senza stato condiviso facilita la divisione del lavoro.

6.2 Lambda, interfacce funzionali e funzioni di ordine superiore

Una **classe anonima** è una classe priva di nome, definita e istanziata nel punto in cui compare una forma estesa dell'operatore `new`; può implementare un'interfaccia per un uso puntuale.

Quando il comportamento richiesto è espresso da un solo metodo astratto, da Java 8 si può usare una **espressione lambda**, cioè una funzione anonima.

Data l'interfaccia `Hello`, con il solo metodo `greetings(String s)`, una lambda può fornire direttamente quel comportamento:

```
Sera sr = new Sera(s -> System.out.println("Ciao, " + s));
sr.saluti();
```

Il costruttore di `Sera` riceve un collaboratore Hello; la lambda fornisce l'operazione di saluto richiesta da quel contratto. Quando il client chiama `saluti()`, l'oggetto usa il collaboratore per eseguire il saluto.

Dal contratto atteso il compilatore ricava che il parametro `s` è una `String`. Questo riconoscimento automatico è detto **inferenza del tipo**. La lambda è possibile perché il comportamento da fornire corrisponde a un solo metodo astratto: non è una sintassi generica per implementare interfacce con più metodi astratti.

La forma generale è

parametri `->` *corpo*. (6.1)

I *parametri*, a sinistra della freccia, sono gli ingressi; il *corpo*, a destra, calcola l'azione e l'eventuale valore restituito. Se il parametro è uno solo si possono omettere sia il suo tipo sia le parentesi; con nessun parametro o con più parametri le parentesi sono necessarie. Un corpo con più istruzioni richiede le parentesi graffe e un `return` esplicito quando deve restituire un valore:

```
s -> System.out.println("Ciao " + s)
(x, y) -> x + y
() -> System.out.println("Buongiorno")
(x, y) -> { System.out.println("x: " + x); return x + y; }
```

Nella seconda lambda, `x` e `y` sono gli ingressi e `x + y` è il risultato.

Una funzione è *pura* se, a parità di ingressi, restituisce lo stesso risultato e non produce effetti osservabili all'esterno del calcolo. Non dipende quindi da stato esterno mutabile, non modifica gli oggetti ricevuti o lo stato condiviso e non esegue operazioni di ingresso/uscita, come stampare sullo schermo o scrivere su un file.

Negli esempi precedenti, `(x, y) -> x + y`, applicata a valori numerici, è pura; la variante che stampa `x` prima di restituire la somma non lo è, pur calcolando lo stesso risultato. L'assenza di campi propri non basta a garantire la purezza.

In Java una lambda può leggere o modificare stato esterno e compiere operazioni di ingresso/uscita: la sintassi lambda non garantisce la purezza. Questi effetti richiedono attenzione, soprattutto in parallelo.

Un'**interfaccia funzionale** è un'interfaccia con un unico metodo astratto. L'annotazione `@FunctionalInterface` chiede al compilatore di verificare questo vincolo e impedisce che

modifiche successive aggiungano per errore un secondo metodo astratto. Una lambda viene interpretata come implementazione dell'unico metodo astratto.

Una **funzione di ordine superiore** riceve una funzione, ne crea una oppure ne restituisce una. Nei metodi degli stream, per esempio, il chiamante passa una lambda per indicare il comportamento da applicare agli elementi.

In Java quel comportamento è rappresentato attraverso un'interfaccia funzionale. Passarlo o restituirlo significa quindi fornire un oggetto compatibile con il contratto atteso: è il collegamento fra la notazione funzionale e il sistema di tipi a oggetti.

Le interfacce funzionali ricorrenti distinguono predicati, trasformazioni, produttori di valori, consumatori e criteri di confronto. I parametri generici `T` e `R` indicano rispettivamente il tipo di ingresso e il tipo di risultato.

Tabella 6.1: Interfacce funzionali usate nelle pipeline.

Interfaccia	Metodo	Significato ed esempio
`Predicate<T>`	`test(T)`	Restituisce un booleano; `x -> x >= 0`.
`Function<T,R>`	`apply(T)`	Trasforma un `T` in un `R`; `x -> x.length()`.
`Supplier<T>`	`get()`	Non riceve parametri e fornisce un valore `T`; `() -> Math.random()`.
`Consumer<T>`	`accept(T)`	Riceve un elemento, compie un'azione e non restituisce valore; è usato da `peek()` e `forEach()`.
`Comparator<T>`	`compare(T,T)`	Stabilisce l'ordine fra due oggetti dello stesso tipo.

6.2.1 Lo stesso comportamento in tre forme

Un oggetto `Sera` riceve il comportamento da eseguire attraverso il contratto `Hello`. Una classe nominata lo realizza così:

```
interface Hello { void greetings(String s); }
class Sera {
    private final Hello myh;
    Sera(Hello h) { myh = h; }
    void saluti() { myh.greetings("buonasera"); }
}
class Saluti implements Hello {
    public void greetings(String s) {
        System.out.println("Ciao, " + s);
    }
}
class MainSal {
    public static void main(String[] args) {
        Sera sr = new Sera(new Saluti());
        sr.saluti();
    }
}
```

Il client dipende da `Sera`; `Sera` conserva `Hello`; `Saluti` implementa `Hello`. La classe anonima elimina il nome della classe concreta, mantenendo lo stesso contratto:

```
Sera sr = new Sera(new Hello() {
    @Override public void greetings(String s) {
        System.out.println("Ciao, " + s);
    }
});
sr.saluti();
```

Con una sola operazione astratta basta la lambda:

```
Sera sr = new Sera(s -> System.out.println("Ciao, " + s));
sr.saluti();
```

Le tre alternative stampano `Ciao, buonasera`. Non si eliminano né il contratto né il collaboratore; si elimina il codice necessario a nominare e dichiarare la classe che realizza un comportamento semplice.

6.2.2 Riferimenti a metodi e a costruttori

La sintassi `::` indica un *riferimento a metodo*: indica un comportamento senza invocarlo immediatamente. `Integer::sum`, per esempio, corrisponde all'addizione `(a, b) -> a + b`; il metodo somma due valori primitivi `int`, con le eventuali conversioni boxing/unboxing richieste dall'interfaccia funzionale. `Persona::getEta` indica invece un metodo da invocare sulla persona ricevuta come argomento. Per gli esempi sulle persone si assumono nome, età e ruolo accessibili tramite `getNome()`, `getEta()` e `getRuolo()`.

Un riferimento può avere la forma `Classe::metodoStatico`, `istanza::metodo` oppure `Classe::metodoDiIstanza`; nell'ultimo caso l'oggetto che riceve la chiamata è fornito fra gli argomenti del comportamento funzionale. La notazione non invoca immediatamente il metodo e deve essere compatibile con il contratto funzionale atteso. Un Supplier conserva un comportamento da eseguire chiamando `get()`, non necessariamente un valore già calcolato: può leggere uno stato aggiornato al momento della chiamata.

`Function<T,R>` definisce `apply(T)`. La funzione `stringLength = x -> x.length()`, con tipo `Function<String,Integer>`, riceve una stringa e ne restituisce la lunghezza: `stringLength.apply("truth")` vale cinque.

Un `Supplier<T>` definisce `get()`, non riceve argomenti e restituisce un `T`: per esempio `Supplier<String> sup = () -> "Ciao ciao";` e `sup.get()` restituisce la stringa. Un riferimento a costruttore, quale `ProdottoA::new`, può essere un `Supplier<Prodotto>`; una mappa da nomi a supplier permette quindi di ritardare la creazione fino alla chiamata a `get()`. Se il costruttore richiede parametri, il solo `Supplier` non è appropriato, perché non ne accetta.

6.3 Collection: strutture dati, costi e origine dello stream

La gerarchia delle *collection* offre strutture dati e operazioni quali `add()`, `remove()`, `size()`, `contains()` e `containsAll()`. `Collection` è l'interfaccia generale; `List` aggiunge, fra gli altri, `get()`, `indexOf()` e `set()`.

- `ArrayList` è un array espandibile: i riferimenti agli elementi sono memorizzati nell'array interno e l'accesso per indice è veloce; un incremento del 50% quando manca spazio è un possibile dettaglio implementativo, non una garanzia del contratto List.

- `LinkedList` è una lista i cui elementi mantengono riferimenti al precedente e al successivo.
- `Vector` è simile a `ArrayList`, ma sincronizza le singole operazioni, con un costo in termini di prestazioni; una sequenza di chiamate non diventa per questo automaticamente atomica.
- `TreeSet` mantiene elementi ordinati e offre operazioni principali con costo logaritmico.

La notazione $O(f(n))$ descrive un limite all'ordine di crescita del costo al crescere degli elementi: $O(1)$ indica costo costante, $O(n)$ lineare e $O(\lg n)$ logaritmico, dove $\lg n$ è il logaritmo di n. Per confrontare le strutture bisogna anche specificare quale operazione si esegue e se la posizione interessata è già nota.

Tabella 6.2: Complessità delle collection: inserimento in coda; * indica posizione già individuata.

Struttura	`get`	`add`	`contains`	`remove`
`ArrayList`	$O(1)$	$O(1)$ amm.	$O(n)$	$O(n)$
`LinkedList`	$O(n)$	$O(1)$*	$O(n)$	$O(1)$*
`TreeSet`	—	$O(\lg n)$	$O(\lg n)$	—

L'aggiunta in coda ad ArrayList ha costo costante *ammortizzato*: alcune aggiunte richiedono di riallocare e copiare l'array, ma il costo medio sulla sequenza di aggiunte resta costante. Inserire o rimuovere nel mezzo richiede spostamenti. In LinkedList il collegamento o scollegamento è costante quando il nodo è già individuato; raggiungerlo per indice o cercarne il valore costa invece $O(n)$.

Java 8 ha introdotto i *metodi di default* nelle interfacce per aggiungere metodi a interfacce esistenti senza rompere la compatibilità con applicazioni precedenti. `stream()` è un metodo di default di `Collection`: su una collection di elementi `T` restituisce `Stream<T>`, una sequenza di elementi `T` su cui applicare operazioni. `List.of(...)` (da Java 9) crea invece una lista non modificabile; `Stream.of(...)` crea direttamente uno stream.

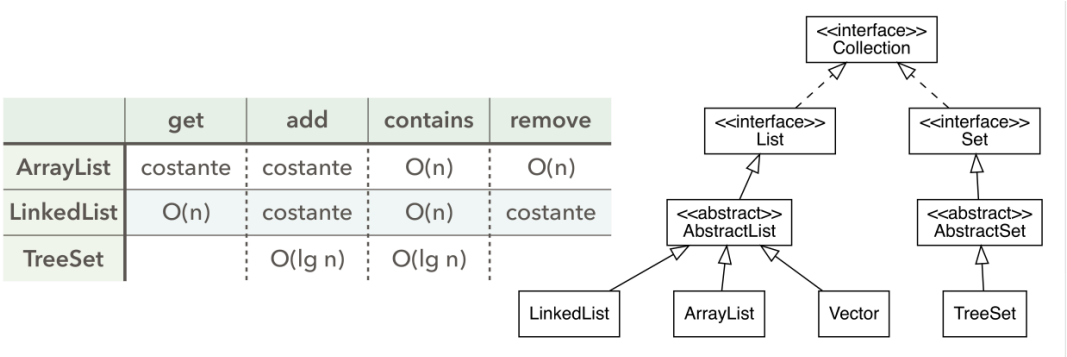


Figura 6.1: Gerarchia Collection/List/Set e confronto dei costi delle operazioni nelle condizioni specificate.

6.4 Pipeline: sorgente, passaggi lazy e operazione terminale

Uno *stream* non è una collection né un contenitore che memorizza dati: è un flusso di elementi provenienti da un'origine, descritto utilmente come un nastro trasportatore.

Una **pipeline** è la catena formata dall'origine, da zero o più operazioni intermedie e da un'operazione terminale. Un'operazione è *intermedia* quando restituisce uno stream; è normalmente *lazy*, cioè la sua elaborazione è rinviata. Operazioni come `filter` e `map` descrivono passaggi successivi dell'elaborazione senza modificare di per sé la collection di origine.

I passaggi sono distinti sul piano logico: non bisogna dedurne che ogni chiamata restituisca un oggetto con identità diversa o crei una copia dei dati. Alcune operazioni possono restituire lo stesso oggetto stream su cui sono invocate. Il riferimento restituito permette di proseguire la pipeline, senza rendere riutilizzabile o duplicare lo stream precedente.

L'operazione *terminale* richiede il valore finale o l'esecuzione dell'azione finale e consuma lo stream. È *eager*: quando viene invocata, forza l'elaborazione delle operazioni precedenti. Senza terminale, la descrizione di una pipeline lazy non viene valutata. Nell'esempio, il filtro descrive la selezione e il conteggio ne richiede il risultato:

```
List<String> nomi = List.of("Nobita", "Nobi", "Suneo");
long c = nomi.stream()
.filter(s -> s.equals("Nobi"))
.count();
```

`filter` seleziona gli elementi per cui il predicato è vero e restituisce uno stream; `count` è terminale e restituisce il numero `long` di elementi selezionati. La pipeline equivale ai passaggi espliciti `s1 = nomi.stream()`, `s2 = s1.filter(...)` e `c = s2.count()`.

Concatenare due `filter` non richiede di costruire prima una lista per il primo filtro e poi una seconda lista per il secondo filtro. In una pipeline, un elemento attraversa i filtri finché viene scartato o raggiunge il terminale.

Con la valutazione lazy la libreria può unire il lavoro di attraversamento ed evitare materializzazioni inutili; non è necessario riscrivere i predicati come un unico AND perché questa proprietà valga. È la libreria a organizzare il passaggio degli elementi; la valutazione lazy non garantisce che il compilatore esegua una specifica riscrittura del codice.

Consumo dello stream ed effetti

Uno stream può essere consumato una sola volta; per una nuova elaborazione si chiede un nuovo stream alla sorgente. `map` ha una corrispondenza uno-a-uno fra elementi attraversati in ingresso e in uscita, pur potendo cambiare il tipo; `filter` seleziona zero oppure un elemento per ciascun ingresso.

Non si modificano automaticamente gli elementi o la collezione di origine, ma una lambda che modifica un oggetto mutabile può comunque produrre effetti osservabili: tale effetto dipende dal codice passato all'operazione.

6.5 Filter e map: selezione e trasformazione

`filter(Predicate<T> p)` riceve un predicato `p`: per ciascun elemento `T`, il risultato di `p.test(...)` è un booleano: se vale `true`, l'elemento prosegue; altrimenti è scartato. Per contare i nomi di cinque caratteri si scrive `nomi.stream().filter(s -> s.length() == 5).count()`; per le stringhe vuote, `filter(s -> s.isEmpty())`.

`map(Function<T,R> mapper)` applica la funzione `mapper` a ogni elemento di tipo `T` e restituisce uno `Stream<R>`. Per esempio `map(x -> x * 2)` raddoppia gli interi; `map(Persona::getEta)` trasforma uno stream di `Persona` in uno stream delle rispettive età. Entrambe sono operazioni intermedie.

6.5.1 Dati completi per trasformazioni e ricerche

Per gli esempi sulle età si usa una classe con costruttore e getter:

```
class Persona {
    private final String nome;
    private final int eta;
    Persona(String nome, int eta) { this.nome = nome; this.eta = eta; }
    public String getNome() { return nome; }
    public int getEta() { return eta; }
    public static Persona getMax(Persona p1, Persona p2) {
        return p1.getEta() > p2.getEta() ? p1 : p2;
    }
}
List<Persona> amici = List.of(new Persona("Saro", 24),
    new Persona("Taro", 21), new Persona("Ian", 19), new Persona("Al", 16));
```

Due filtri concatenati `filter(p -> p.getNome().equals("Taro"))` e `filter(p -> p.getEta()==21)` selezionano Taro. Cambiare l'età di Al da 16 a 21 in un'altra fixture modifica i dati, pur lasciando invariato questo risultato. Una ricerca su sole stringhe, come `nomi.contains("Saro")`, controlla invece soltanto il nome e non una proprietà della persona.

Un predicato dichiarato separatamente può essere applicato a una sorgente concreta:

```
Predicate<Integer> positive = x -> x >= 0;
List<Integer> nonNegativi = Stream.of(2, 5, 10, -1)
    .filter(positive).toList(); // [2, 5, 10]
List<Integer> eta = new ArrayList<>();
for (Persona p : amici) eta.add(p.getEta());
```

Il nome `positive` non cambia la condizione: qui è incluso anche zero. L'ultimo ciclo costruisce la stessa sequenza di età di `map(Persona::getEta).collect(Collectors.toList())`, cioè 24, 21, 19, 16. `Stream` è un'interfaccia: il client ne usa il contratto senza nominare le classi interne che realizzano filtri, trasformazioni e riduzioni.

La selezione di valori maggiori di cinque può produrre risultati diversi senza cambiare il predicato: `numeri.stream().filter(n -> n>5).count()` restituisce un conteggio, mentre `numeri.stream().filter(n -> n>5).toList()` restituisce i valori selezionati. L'operazione terminale determina il tipo di risultato; per ogni nuova elaborazione occorre uno stream nuovo.

6.6 Collect e toList: materializzare il risultato

Il metodo `collect()` raccoglie gli elementi finali secondo un `Collector`. Il collector `Collectors.toList()` li accumula in una `List`. Da Java 16 è disponibile anche `Stream.toList()`. Un esempio completo ricava le età:

```
List<Integer> eta = amici.stream()
.map(Persona::getEta)
.collect(Collectors.toList());
```

`Stream.toList()` restituisce una lista non modificabile. `Collectors.toList()` non garantisce né una particolare implementazione concreta né uno specifico contratto di mutabilità. Per ottenere un ArrayList modificabile si specifica il contenitore:

```
.collect(Collectors.toCollection(ArrayList::new))
```

Gli elementi contenuti non diventano per questo immutabili.

6.7 Reduce: accumulazione, identità e associatività

Una **riduzione** combina molti elementi in un solo valore. Nella forma `reduce(T identity, BinaryOperator<T> accumulator)`, il chiamante fornisce un valore iniziale, l'identità `identity`, dello stesso tipo degli elementi, e un accumulatore binario. Quest'ultimo combina il risultato parziale con il prossimo elemento.

La formula descrive questo avanzamento. Dato lo stream finito $v_1, \ldots, v_n$, si parte da $a_0 = I$, dove I è l'identità; a ogni passo si applica l'accumulatore g:

$$a_i = g(a_{i-1}, v_i) \quad (1 \le i \le n), \qquad \text{risultato} = a_n. \tag{6.2}$$

Qui v_i è l'i-esimo elemento e a_i il valore accumulato dopo averlo elaborato. Per la somma, $I = 0$ e $g(a, v) = a + v$: `reduce(0, (accum, v) -> accum + v)`. Se lo stream è vuoto, questa variante restituisce l'identità, quindi 0 nell'esempio. `reduce` è terminale.

La variante `reduce(BinaryOperator<T>)` non riceve identità e restituisce `Optional<T>`, perché non esiste un risultato da estrarre quando lo stream è vuoto. `Optional<T>` rappresenta un valore di tipo `T` che può essere presente oppure assente; si può usare `isPresent()` per verificare la presenza e `get()` per estrarre il valore solo nel ramo in cui è presente.

Identità e raggruppamento in parallelo

La formula della riduzione precedente descrive l'accumulazione sequenziale. In parallelo, gli elementi possono essere raggruppati e combinati in modo diverso: l'operatore deve essere associativo e l'identità deve essere neutra. Per la somma, $0 + x = x$; scegliere 10 come presunta identità può aggiungere più volte quel valore quando si combinano partizioni. La sottrazione non è associativa: $(a - b) - c$ e $a - (b - c)$ possono differire.

6.7.1 Esempi di riduzione

`Function<T,R>` definisce `apply(T)`: riceve un oggetto di tipo `T` e restituisce un risultato di tipo `R`. Nell'esempio seguente `stringLength` calcola la lunghezza di una stringa; passato a `map`, consente di sommare le lunghezze e di ottenere 31.

```
Function<String, Integer> stringLength = x -> x.length();
int totale = Stream.of("truth", "flows", "to", "them",
                       "sweetly", "by", "nature")
.map(stringLength).reduce(0, Integer::sum);
```

6.7.2 Esempio: somma delle età

Per una lista `amici` di `Persona`, la somma delle età è una trasformazione seguita da una riduzione:

```
int somma = amici.stream()
.map(Persona::getEta)
.reduce(0, Integer::sum);
```

Il primo passaggio produce le età, il secondo le combina a partire dall'identità zero.

Con la fixture delle quattro persone, le tre forme equivalenti della somma sono:

```
int sommaImperativa = 0;
for (Persona p : amici) sommaImperativa += p.getEta();
int sommaLambda = amici.stream().map(p -> p.getEta())
    .reduce(0, (acc, eta) -> acc + eta);
int sommaMetodo = amici.stream().map(Persona::getEta)
    .reduce(0, Integer::sum);
```

Tutte restituiscono 80. Il primo stile controlla l'iterazione e aggiorna una variabile; gli altri descrivono trasformazione e combinazione. Per gli importi `List<Float> valori`, lo stesso confronto dà:

```
float totale = 0;
for (float v : valori) totale += v;
float ridotto = valori.stream().reduce(0f, Float::sum);
```

Sono due corpi alternativi di `calcolaSommaImper` e `calcolaSomma`. La virgola mobile può rendere significativo il raggruppamento delle somme: non si deduce l'identità bit per bit fra qualsiasi esecuzione parallela e sequenziale.

6.8 Comparator, max e sorted: confronto e ordine

Un `Comparator<T>` definisce il criterio con cui ordinare gli oggetti. Il metodo `compare(o1,o2)` restituisce 0 se `o1` e `o2` sono equivalenti secondo quel criterio, un valore positivo se `o1` è maggiore di `o2`, negativo nel caso opposto.

Per costruire il confronto per età, `Comparator.comparing(Persona::getEta)` estrae da ogni persona l'età e la usa come chiave. Lo stesso criterio serve a due operazioni con risultati diversi: `max(comparator)` è terminale e restituisce il massimo in un `Optional`; `sorted(comparator)` è intermedia e restituisce gli elementi ordinati, sui quali la pipeline può proseguire.

Un Comparator restituisce zero quando i due elementi sono equivalenti secondo il criterio di confronto, anche se non sono lo stesso riferimento e non sono uguali secondo `equals`. Se si confrontano le età con `p1.getEta()-p2.getEta()`, per 10 e 25 il risultato è -15, coerentemente con il fatto che dieci precede venticinque in ordine crescente.

Per evitare anche i problemi di overflow di una sottrazione fra interi generici, si può scrivere:

```
Comparator<Persona> perEta =
    Comparator.comparingInt(Persona::getEta);
```

`max(perEta)` restituisce la Persona massima secondo quel criterio, non direttamente l'età; per ottenere il valore si trasforma prima lo stream o si legge la Persona risultante dall'Optional.

```
Optional<Persona> pmax = amici.stream()
.filter(p -> p.getEta() < 20)
.max(Comparator.comparing(Persona::getEta));
if (pmax.isPresent())
    System.out.println(pmax.get().getNome());
```

La versione imperativa del massimo sotto i vent'anni mantiene un candidato inizialmente assente:

```
Persona candidato = null;
for (Persona p : amici) {
    if (p.getEta() < 20
            && (candidato == null || p.getEta() > candidato.getEta()))
        candidato = p;
}
if (candidato != null) System.out.println(candidato.getNome());
Optional<Persona> conReduce = amici.stream()
    .filter(p -> p.getEta() < 20).reduce(Persona::getMax);
conReduce.ifPresent(p -> System.out.println(p.getNome()));
```

La riduzione senza identità rappresenta l'assenza con `Optional`; il metodo statico `getMax` sceglie fra due persone. Con Ian 19 e Al 16 il risultato è Ian; se nessuno passa il filtro, non si stampa alcun nome. `max(Comparator)` esprime direttamente lo stesso obiettivo.

In caso di pari età occorre distinguere massimo rispetto al criterio e identità della persona selezionata: una politica specifica di spareggio richiede un criterio ulteriore.

Il metodo di ordinamento di Stream si chiama `sorted()`, mentre `sort()` appartiene ad altri contesti, per esempio alla lista mutabile.

6.8.1 Esempio: selezione e ordinamento delle persone

Per il team si usa una definizione alternativa del tipo Persona: un record con nome, età e ruolo. Gli accessori diventano `nome()`, `eta()` e `ruolo()`, senza prefisso `get`.

```
record Persona(String nome, int eta, String ruolo) { }
List<Persona> team = List.of(new Persona("Kent",29,"CTO"),
    new Persona("Luis",25,"Programmer"), new Persona("Al",25,"Programmer"),
    new Persona("Andrea",26,"GroupLeader"), new Persona("Joe",26,"Programmer"));
team.stream().filter(p -> p.eta() < 26)
```

```
    .filter(p -> p.ruolo().equals("Programmer"))
    .sorted(Comparator.comparing(Persona::nome))
    .forEach(p -> System.out.print(p.nome() + " "));
```

L'output è `Al Luis`: Joe ha 26 anni e viene escluso dalla condizione stretta. I due filtri e l'ordinamento sono intermedi; `forEach` è il terminale.

6.9 Distinct, findFirst, findAny e limit

`distinct()` elimina i duplicati secondo l'uguaglianza degli elementi. È intermedia: produce ancora uno stream.

Per estrarre un elemento si possono usare `findAny()` e `findFirst()`. Sono terminali, restituiscono `Optional` e possono interrompere l'esplorazione senza esaminare l'intero stream: questa proprietà è detta *short-circuiting*.

`limit(k)` tronca invece lo stream a k elementi, dove k è il limite richiesto. Rimane un'operazione intermedia: limita gli elementi che proseguiranno, senza estrarre da sola il risultato finale.

`findFirst()` rispetta l'ordine d'incontro quando esiste; `findAny()` può scegliere un elemento qualsiasi. La differenza è rilevante in parallelo, ma non va intesa come promessa che una specifica istanza di thread vinca sempre. `limit(1)` restituisce ancora uno stream, non l'elemento né un Optional; occorre un'operazione terminale.

Inoltre `limit(k)` pone un massimo, non garantisce k risultati se a monte ce ne sono meno.

6.9.1 Esempio: ruoli distinti

I ruoli diversi sono ottenuti proiettando ogni persona sul ruolo e rimuovendo i duplicati:

```
team.stream().map(Persona::ruolo).distinct()
.forEach(r -> System.out.print(r + " "));
```

Con il team appena definito si stampa `CTO Programmer GroupLeader`, nell'ordine del primo incontro dei ruoli. La ricerca di un Programmer può essere espressa come:

```
Optional<Persona> qualsiasi = team.stream()
    .filter(p -> p.ruolo().equals("Programmer")).findAny();
qualsiasi.ifPresent(p -> System.out.println(p.nome()));
```

Luis, Al e Joe sono tutti risultati ammessi; `findAny` non ne impone uno fisso. `findFirst` sullo stesso stream ordinato per incontro seleziona Luis. Per una ricerca con due requisiti diversi, un tipo `PersonaAlta(String nome,int altezza)` permette `filter(p -> p.nome().equals("Mario"))`, seguito da `filter(p -> p.altezza()>=170)` e da `findAny()` o `findFirst()`: l'Optional è vuoto se non esiste una persona che soddisfa entrambe le condizioni.

6.10 Operazioni stateless e stateful

Un'operazione intermedia **stateless** può elaborare ogni elemento senza conservare informazioni sugli altri: `map` e `filter` ne sono esempi. Un'operazione intermedia **stateful** richiede invece

informazioni sull'insieme degli elementi: `sorted` deve ordinarli e `distinct` deve riconoscere quelli già incontrati. Lo stato necessario può quindi crescere con la dimensione dello stream.

Stato intermedio e accumulatore terminale

Le riduzioni terminali hanno comunque un accumulatore. Per somma (`reduce`) e massimo (`max`), la dimensione del risultato parziale può restare costante anche quando aumenta il numero di elementi; una riduzione che costruisce una struttura crescente può richiedere sempre più memoria. Conservare un accumulatore in una riduzione terminale e classificare un'operazione intermedia come stateful sono quindi due aspetti diversi.

6.11 Generare sequenze con iterate e generate

`Stream.iterate(seme, f)` genera uno stream infinito ordinato: produce il `seme`, poi f(`seme`), poi $f(f$(`seme`)), e così via. Ogni valore dipende dal precedente; la generazione presenta quindi una dipendenza sequenziale, anche se la pipeline può essere configurata come parallela. Occorre limitarlo quando si desidera un risultato finito:

```
Stream.iterate(2, n -> n * 2)
.limit(10)
.forEach(System.out::println); // 2, 4, 8, ..., 1024
```

`Stream.generate(supplier)` genera uno stream infinito: il Supplier fornisce ogni valore senza ricevere quello precedente. Per esempio, `Stream.generate(Math::random).limit(5)` produce cinque `double` casuali maggiori o uguali a zero e minori di uno. Un generatore che arrotonda il valore scalato può essere definito così:

```
Supplier<Long> intero = () -> Math.round(Math.random() * 10);
```

I valori ottenuti sono interi di tipo `long` fra zero e dieci, estremi inclusi.

Posizione del limite e terminazione

Per uno stream infinito, la posizione del limite conta:

```
Stream.generate(() -> 0)
.limit(10).filter(n -> n < 0).count(); // termina: 0
// Con filter(n -> n < 0) prima di limit(10),
// non arriverebbe mai un elemento al limite.
```

Una generazione illimitata non garantisce che il terminale finisca soltanto perché esiste un `limit` dopo un filtro che non produce risultati.

6.12 Debug della pipeline con peek

`forEach` è terminale: esegue un'azione per ogni elemento e non permette di proseguire la pipeline. Per osservare i valori intermedi in fase di debug si usa `peek(Consumer<T>)`: riceve un elemento, compie per esempio una stampa, non ne modifica il passaggio e restituisce lo stream. È lazy: senza un terminale non viene eseguito.

```
List<Integer> numbers = List.of(2, 3, 4, 5);
List<Integer> risultato = numbers.stream().map(x -> x + 17)
    .peek(x -> System.out.println("after map: " + x))
    .filter(x -> x % 2 == 0)
    .peek(x -> System.out.println("after filter: " + x))
    .limit(3)
    .peek(x -> System.out.println("after limit: " + x)).toList();
```

La traccia sequenziale è `after map: 19`; poi, per 20, `after map`, `after filter` e `after limit`; poi `after map: 21`; infine i tre passaggi per 22. Il risultato è `[20,22]`; il limite 3 non crea un terzo valore. Il processamento procede elemento per elemento attraverso la pipeline, non completando prima un'intera operazione su tutti gli elementi.

Precisazione

`peek()` serve a osservare la pipeline durante il debug. Il corto circuito e le ottimizzazioni possono far sì che non tutti gli elementi lo attraversino: non si inseriscono in questa operazione aggiornamenti indispensabili alla correttezza. Per osservare più fasi si colloca un peek dopo ciascuna trasformazione di interesse, conservando l'assenza di effetti sullo stato applicativo.

6.13 Stream primitivi e conversioni

`IntStream` è uno stream del tipo primitivo `int`. `IntStream.rangeClosed(a,b)` genera gli interi da a a b, estremi inclusi, con incremento uno. Perciò il calcolo seguente restituisce $1+2+3+4+5=15$:

```
int somma = IntStream.rangeClosed(1, 5).sum();
```

`mapToInt` trasforma uno stream generico in `IntStream`; `boxed` esegue la conversione opposta, da `IntStream` a `Stream<Integer>`; `mapToObj` crea uno stream di oggetti a partire da un `IntStream`.

Le specializzazioni standard sono `IntStream`, `LongStream` e `DoubleStream`; non esiste `FloatStream` nell'API standard. `IntStream` non estende `Stream<Integer>`: entrambi fanno parte della famiglia degli stream, ma hanno operazioni e tipi diversi. `range(a,b)` esclude il limite superiore, `rangeClosed(a,b)` lo include.

Tre conversioni concrete collegano i contratti:

```
IntStream.rangeClosed(1,6).map(x -> x*x).forEach(System.out::println);
int lunghezze = Stream.of("truth","flows","to","them","sweetly","by","nature")
    .mapToInt(String::length).sum(); // 31
List<Integer> dieci = IntStream.rangeClosed(1,10).boxed().toList();
List<Persona> primiQuattro = IntStream.rangeClosed(0,3)
    .mapToObj(i -> amici.get(i)).toList();
```

Si stampano i quadrati 1, 4, 9, 16, 25, 36. La selezione per indice richiede che `amici` contenga almeno quattro elementi; generare gli indici non verifica automaticamente che rientrino nei limiti della lista.

6.13.1 Esempio: verificare l'ordinamento di una lista

Per verificare se una lista di interi è ordinata si generano gli indici delle coppie contigue, si cerca una violazione e si nega la sua presenza:

```
boolean ordinata = !IntStream.rangeClosed(0, lista.size() - 2)
.filter(i -> lista.get(i) > lista.get(i + 1))
.findAny().isPresent();
```

L'indice `i` varia da 0 a `lista.size()` − 2, così `i+1` resta un indice valido. `findAny` può fermarsi alla prima coppia non ordinata.

6.13.2 Esempio: terne contigue che formano triangoli

Data una lista di numeri interi positivi, una terna (a, b, c) forma un triangolo se ciascun lato è minore della somma degli altri due:

$$a < b + c \ \wedge \ b < a + c \ \wedge \ c < a + b, \tag{6.3}$$

dove a, b e c sono le tre lunghezze positive. Per le terne contigue di una lista si generano gli indici iniziali, si costruisce la terna e si applicano le tre selezioni:

```
IntStream.rangeClosed(0, lista.size() - 3)
.mapToObj(i -> new int[]{lista.get(i), lista.get(i+1), lista.get(i+2)})
.filter(t -> t[0] < (long) t[1] + t[2])
.filter(t -> t[1] < (long) t[0] + t[2])
.filter(t -> t[2] < (long) t[1] + t[0])
.forEach(t -> System.out.println(t[0] + " " + t[1] + " " + t[2]));
```

La promozione a `long` prima della somma evita overflow nell'addizione di due interi positivi. La lista $\{2, 3, 5, 7, 8\}$ produce le terne valide `3 5 7` e `5 7 8`. L'uso di `rangeClosed` fino a `size()-3` assicura che gli accessi `i`, `i+1`, `i+2` esistano.

6.14 Esercizio svolto: buste paga e raggruppamento per ruolo

Si deve calcolare il pagamento mensile di sei lavoratori, il totale e un elenco per ruolo. La tariffa dipende dal ruolo e viene moltiplicata per 25; in mancanza di una tariffa associata si usa 50. I record appartengono a questo esempio e usano accessori senza prefisso `get`:

```
import java.util.*;
import static java.util.Comparator.comparing;

record Persona(String nome, int eta, String ruolo) { }
record BustaPaga(Persona dipendente, int totale) { }
class ListaPersone {
    private final List<Persona> team = List.of(
        new Persona("K Gray", 29, "CTO"),
        new Persona("L Rose", 25, "Programmer"),
        new Persona("Al White", 25, "Programmer"),
        new Persona("Joe Black", 26, "Programmer"),
        new Persona("Jill Purple", 26, "Tester"),
        new Persona("Bill Green", 26, "Analyst"));
    private final Map<String, Integer> roleCost = Map.of(
```

```
        "Programmer", 50, "CTO", 65, "Tester", 56, "Analyst", 54);
    private List<BustaPaga> buste = List.of();
    public void genListaPagamenti() {
        buste = team.stream()
            .map(p -> new BustaPaga(p,
                roleCost.getOrDefault(p.ruolo(), 50) * 25))
            .sorted(comparing(b -> b.dipendente().nome())).toList();
    }
    public int calcolaSommaBuste() {
        return buste.stream().map(BustaPaga::totale)
            .reduce(0, Integer::sum);
    }
    public void stampaBuste() {
        buste.forEach(b -> System.out.println(
            b.dipendente().nome() + "\t" + b.totale()));
    }
    public void stampaRuoliPersone() {
        roleCost.keySet().stream().sorted()
            .peek(System.out::println)
            .forEach(ruolo -> team.stream()
                .filter(p -> p.ruolo().equals(ruolo))
                .sorted(comparing(Persona::nome))
                .forEach(p -> System.out.println("  " + p.nome())));
    }
}
```

Prima di sommare o stampare le buste si chiama `genListaPagamenti`. `map` produce un nuovo record per lavoratore; `sorted` ordina per nome della persona contenuta nel record e `toList` materializza il risultato.

La riduzione successiva somma soltanto i totali.

Si ottiene:

Dipendente	Pagamento
Al White	1250
Bill Green	1350
Jill Purple	1400
Joe Black	1250
K Gray	1625
L Rose	1250
Totale	8125

`stampaRuoliPersone` visita le chiavi delle tariffe in ordine alfabetico: Analyst, CTO, Programmer, Tester. Per ciascuna stampa l'intestazione e avvia una seconda pipeline che seleziona e ordina i lavoratori di quel ruolo. Il `peek` è qui una traccia del flusso per gruppi; se l'intestazione è un effetto richiesto dall'applicazione è più chiaro stamparla nel corpo del `forEach`, insieme alla pipeline interna.

Un ruolo non presente nella mappa riceve comunque tariffa 50 nel calcolo, ma non compare in questa stampa basata sulle sole chiavi della mappa: per elencare tutti i ruoli effettivi la sorgente deve derivare dai ruoli del team con `distinct()`.

6.15 Stream paralleli: condizioni di correttezza e prestazioni

La programmazione parallela cerca di distribuire il lavoro su più core. Rispetto all'esecuzione sequenziale aggiunge un problema di coordinamento: con `Thread` occorre in generale gestire condizioni di corsa, cioè comportamenti che dipendono dall'interazione temporale fra attività concorrenti, e strumenti quali `synchronized`, `wait`, `notify`, lock ed esecutori.

`Collection.parallelStream()` può affidare alla libreria la ripartizione del lavoro in blocchi tramite uno stream parallelo; `stream()` produce la variante sequenziale. La libreria organizza il lavoro, mentre il chiamante deve rispettare le condizioni sulle operazioni e sullo stato esposte di seguito.

Le operazioni senza effetti sullo stato condiviso e l'assenza di modifiche alla sorgente facilitano la parallelizzazione. Una lambda che aggiorna uno stato globale richiede invece coordinamento e può compromettere correttezza e prestazioni. Su milioni di elementi e hardware multicore, sorgenti come ArrayList, Vector e TreeSet possono beneficiare della divisione del lavoro; LinkedList può offrire vantaggi inferiori, perché la rappresentazione concatenata è meno agevole da dividere in parti indipendenti.

Il risultato dipende dall'operazione e dalla macchina, e `forEach` parallelo non garantisce l'ordine di esecuzione.

```
long c = nomi.parallelStream()
.map(String::toUpperCase)
.filter(s -> s.equals("NOBI"))
.count();
```

Precisazione

Il lavoro parallelo richiede di suddividere, coordinare e ricomporre il lavoro: per pochi elementi il costo aggiuntivo di queste operazioni, cioè l'overhead, può superare il guadagno.

Un cambiamento del JDK può modificare le prestazioni senza cambiare l'API, ma non garantisce sempre un miglioramento. Una sorgente facilmente partizionabile e operazioni senza stato condiviso aiutano; ordinamento, duplicati e ordine richiesto possono aumentare il costo.

6.15.1 Leggere un confronto sperimentale

Due esperimenti confrontano ricerca su cinque milioni di elementi con ciclo esterno, stream sequenziale e stream parallelo, usando ArrayList, Vector, LinkedList e TreeSet. Il primo impiega hardware a quattro core e Java 11.0.2; il secondo hardware a otto core e Java 18.0.1. I grafici inferiori mostrano anche il costo di riempimento delle strutture.

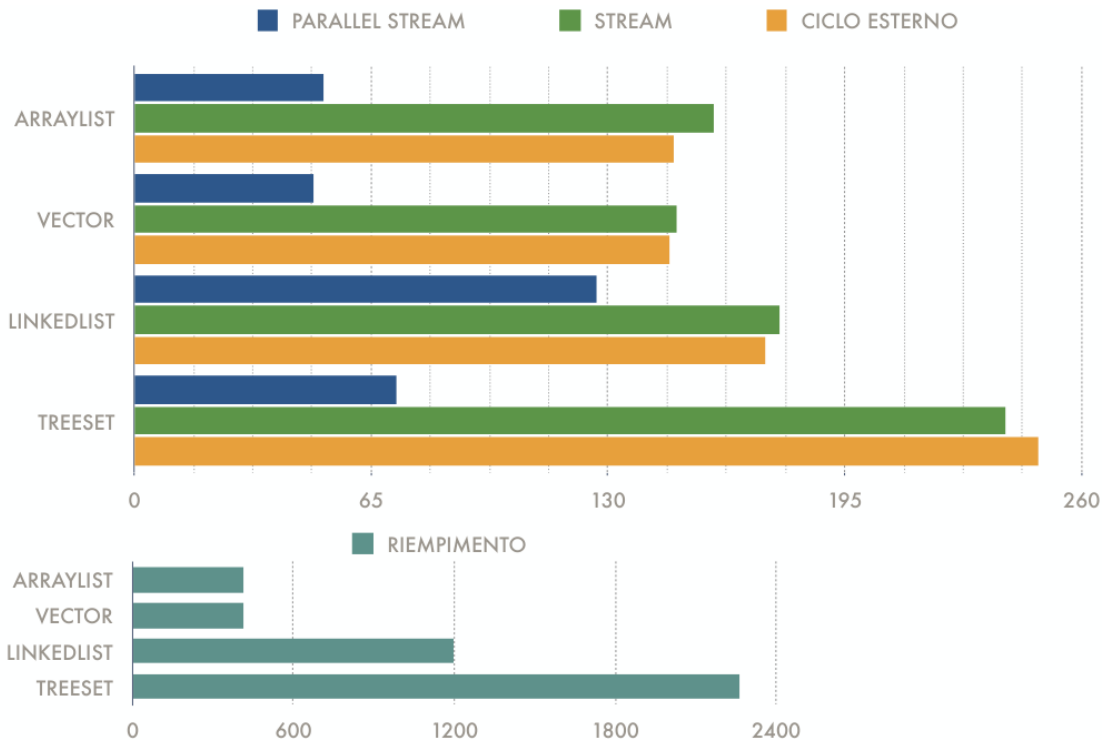


Figura 6.2: Ricerca su cinque milioni di elementi e riempimento: quattro core, Java 11.0.2.

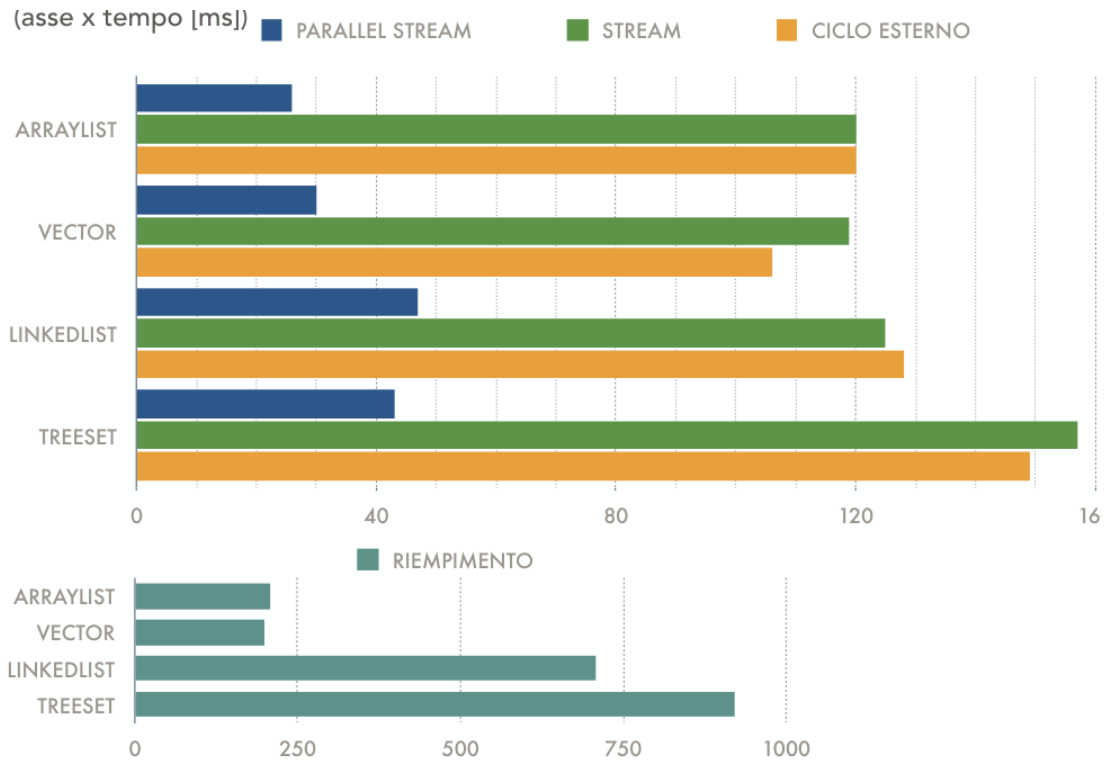


Figura 6.3: Ricerca su cinque milioni di elementi e riempimento: otto core, Java 18.0.1; tempi in millisecondi.

Nel primo confronto il parallelo riduce sensibilmente il tempo di ricerca su ArrayList, Vector e TreeSet, con un beneficio minore su LinkedList.

Nel secondo il parallelo resta il più rapido nei casi illustrati; i tempi di riempimento delle strutture restano diversi. Le barre consentono un confronto qualitativo e una lettura approssimativa dell'asse, non misure esatte dei singoli casi.

Cambiano insieme hardware e JDK: non si può attribuire la differenza fra i due esperimenti al solo hardware o al solo JDK, né trasformarla in una garanzia per ogni pipeline.